



PES Institute of Technology, Bangalore
Department of Computer Science / Information Science & Engineering
SEMESTER END EXAMINATION (SEE) B. E. VI SEMESTER
[Session: May, 2017]

14CS351

14CS351 - COMPILER DESIGN

Time: 3 hrs.

Answer All Questions

Max Marks: 100

1.	a)	Consider the following tokens and their associated regular expressions, given as a lex-like specification: %% (01 10) print ("course") 0(01) *1 print ("compiler") (1010*1 0101*0) print ("design") Give an input to this scanner such that the output string is
----	----	--

		I. Attempt to show a shift-reduce parse of the string bbbba for a parser for grammar G_1. Show the contents of the stack, the input, and the actions (i.e., shift, reduce, error, accept). You don't need to create a parse table; just use your knowledge of the grammar and how the parser works. Be sure to indicate any conflicts and explain why they are conflicts. II. Using the definition and properties of various parsers that you studied in Compiler design course, justify whether or not: 1. G_1 is LR(1)? 2. G_2 is LR(0)? 3. G_1 is LL(1)? : 4. G_2 is LL(1)? [You get credit only for correct justification not for writing yes or No]	
2.	b)	Consider the following grammar over the alphabet $\Sigma = \{u, v, w, x, y, z\}$ $ \begin{aligned} S &\rightarrow UVW \\ U &\rightarrow u \mid Wv \mid \epsilon \\ V &\rightarrow w \mid \epsilon U \mid \epsilon \\ W &\rightarrow y \mid z \end{aligned} $ I. Give the first sets and follow sets of each non-terminal in the grammar. II. Is this grammar LL (1)? Justify.	10
3.	a)	Consider grammar and rules given below for array address translation and generating 3 address code for array references: • L.array.basname means name of the array • L.array.typeofelement means type of the element of the array • L.type.width means width of L.type Assume size of integer to be 4 bytes, and lower bound of the arrays to be 0 Let A, B and C be 10 X 5, 5 X 7, and 10 X 7 arrays of integers respectively. Let i, j, and k be integers. Construct an annotated parse tree for the expression $C[i][j] + A[i][k] * B[k][j]$ and show the 3-address code sequence generated for the expression. $ \begin{aligned} E &\rightarrow E_1 + E_2 \{ E.addr = newtemp(); \\ &\quad gen(E.addr '=' E_1.addr '+' E_2.addr); \} \\ E &\rightarrow E_1 * E_2 \{ E.addr = newtemp(); \\ &\quad gen(E.addr '=' E_1.addr '*' E_2.addr); \} \\ id &\quad \{ E.addr = id.lexeme; \} \\ L &\quad \{ E.addr = newtemp(); \\ &\quad gen(E.addr '=' L.array.basname '[' L.addr ']'); \} \\ L &\rightarrow id [E] \{ L.array = id.lexeme; L.type = L.array.typeofelement; \\ &\quad L.addr = newtemp(); \\ &\quad gen(L.addr '=' E.addr '*' L.type.width); \} \\ L_1 [E] &\quad \{ L.array = L_1.array; L.type = L_1.type.typeofelement; \\ &\quad t = newtemp(); L.addr = newtemp(); \\ &\quad gen(t '=' E.addr '*' L.type.width); \\ &\quad gen(L.addr '=' L_1.addr '+' t); \} \end{aligned} $	10

3.	b)	Provide an implementation of SDT scheme for simple type declaration (grammar is given below for reference) during LR parsing. For full credit, provide proper explanation and show parser stack when necessary. Consider for example the input string as <code>int a, b</code> $D \rightarrow TL$ $T \rightarrow \text{int} \mid \text{float}$ $L \rightarrow L, \text{id} \mid \text{id}$ (Note: Provide the general structure of parser stack used during LR parsing.)	10											
4.	a)	What are the different ways in which a procedure can access non-local data on a run time stack? Explain each technique using a simple example.	5											
4.	b)	Generate Target Code for the following procedure call assuming static allocation. <table border="1"><thead><tr><th></th><th>Code is kept at address</th><th>Activation record is kept at address</th></tr></thead><tbody><tr><td>main</td><td>100</td><td>600</td></tr><tr><td>p</td><td>400</td><td>800</td></tr></tbody></table> <table border="1"><tbody><tr><td><pre>//main() n = 6 i = 0 L1 : if i >= 6 goto L2 i = i + 1 goto L1 L2 : call p halt</pre></td><td><pre>//p() i = 60 return</pre></td></tr></tbody></table>		Code is kept at address	Activation record is kept at address	main	100	600	p	400	800	<pre>//main() n = 6 i = 0 L1 : if i >= 6 goto L2 i = i + 1 goto L1 L2 : call p halt</pre>	<pre>//p() i = 60 return</pre>	5
	Code is kept at address	Activation record is kept at address												
main	100	600												
p	400	800												
<pre>//main() n = 6 i = 0 L1 : if i >= 6 goto L2 i = i + 1 goto L1 L2 : call p halt</pre>	<pre>//p() i = 60 return</pre>													
4.	c)	Generate 3-address code for the following program and convert it into a CFG. <pre>int prime(int n) { int n, i, flag = 0; for(i=2; i<=n/2; ++i) { if(n%i==0){ flag=1; break; } } if (flag==0) return 0; else return 1; }</pre>	10											
5.	a)	What are the issues in the design of a code generator?	5											
5.	b)	Optimize the code below by applying the following code transformations: constant propagation, constant folding, copy-propagation, dead-code elimination and strength reduction.	5											

		<pre>L0: t1 = t1 + 1 t2 = 0 t3 = t1 * 8 t4 = t3 + t2 t5 = t4 * 4 t6 = *t5 t7 = FP + t3 *t7 = t2 t8 = t1 if (t8 > 0) goto L1 L1: goto L0 L2: t1 = 1 t10 = 16 t11 = t1 * 2 goto L1</pre>																															
5.	c)	Perform Live variable analysis on the following CFG. Provide your answer in the table format as given below next to the CFG. <pre>graph TD B1[B1: y=17] --> B2[B2: x=read()] B2 --> B3[B3: x=clean(x)] B2 --> B4[B4: z=x+y] B3 --> B4 B4 --> B5[B5: y=x+1]</pre> <table><thead><tr><th></th><th>use</th><th>def</th><th>IN</th><th>OUT</th></tr></thead><tbody><tr><td>B1</td><td></td><td></td><td></td><td></td></tr><tr><td>B2</td><td></td><td></td><td></td><td></td></tr><tr><td>B3</td><td></td><td></td><td></td><td></td></tr><tr><td>B4</td><td></td><td></td><td></td><td></td></tr><tr><td>B5</td><td></td><td></td><td></td><td></td></tr></tbody></table> Note : clean(x) is some function performed on the value of x. <i>read() reads the value of x from the user.</i>		use	def	IN	OUT	B1					B2					B3					B4					B5					5
	use	def	IN	OUT																													
B1																																	
B2																																	
B3																																	
B4																																	
B5																																	
5.	d)	Construct DAG for the following Block and optimize. <pre>1. t1 := 4 * I 2. t2 := A - 4 3. t3 := t2 [t1] 4. t4 := 4 * I 5. t5 := B - 4 6. t6 := t5 [t4] 7. t7 := t3 * t6 8. t8 := PROD + t7 9. PROD := t8 10. t9 := I + 1 11. I = t9 12. if I ≤ 20 goto (1).</pre>	5																														